Quickcheck
Hackage: https://hackage.haskell.org/package/QuickCheck
Manual: https://www.cse.chalmers.se/~rjmh/QuickCheck/manual.html

Test.Quickcheck

```
quickCheck :: Testable prop => prop -> IO ()
data Args = Args {

    replay :: Maybe (QCGen, Int)
    maxSuccess :: Int
    maxDiscardRatio :: Int
    maxSize :: Int
    chatty :: Bool
    maxShrinks :: Int

}

data Result
    = Success {
        numTests :: Int
        numDiscarded :: Int
        labels :: !(Map [String] Int)
        classes :: !(Map String Int)
        tables :: !(Map String (Map String Int))
        output :: String
    }
    | GaveUp {
        numTests :: Int
        numDiscarded :: Int
        labels :: !(Map [String] Int)
        classes :: !(Map String Int)
        tables :: !(Map String (Map String Int))
        output :: String
    }
    | Failure {
        numTests :: Int
        numDiscarded :: Int
        numShrinks :: Int
        numShrinkTries :: Int
        numShrinkFinal :: Int
        usedSeed :: QCGen
        usedSize :: Int
        reason :: String
        theException :: Maybe AnException
        output :: String
        failingTestCase :: [String]
        failingLabels :: [String]
        failingClasses :: Set String
        witnesses :: [Witness]
    }
    | NoExpectedFailure {
        numTests :: Int
        numDiscarded :: Int
        labels :: !(Map [String] Int)
```

```
        classes :: !(Map String Int)
        tables :: !(Map String (Map String Int))
        output :: String
    }

data TestProgress = TestProgress {
    currentPassed :: Int
    currentDiscarded :: Int
    maxTests :: Int
    currentShrinks :: Int
    currentFailedShrinks :: Int
    currentTotalShrinks :: Int
}
stdArgs :: Args

quickCheckWith :: Testable prop => Args -> prop -> IO ()
quickCheckWithResult :: Testable prop => Args -> prop ->
IO Result
quickCheckResult :: Testable prop => prop -> IO Result
recheck :: Testable prop => Result -> prop -> IO ()
isSuccess :: Result -> Bool
verboseCheck :: Testable prop => prop -> IO ()
verboseCheckWith :: Testable prop => Args -> prop -> IO
()
verboseCheckWithResult :: Testable prop => Args -> prop
-> IO Result
verboseCheckResult :: Testable prop => prop -> IO Result
quickCheckAll :: Q Exp
verboseCheckAll :: Q Exp
forAllProperties :: Q Exp
allProperties :: Q Exp
polyQuickCheck :: Name -> ExpQ
polyVerboseCheck :: Name -> ExpQ
monomorphic :: Name -> ExpQ

class Arbitrary a where
    arbitrary :: Gen a
    shrink :: a -> [a]

genericShrink :: (Generic a, RecursivelyShrink (Rep a),
GSubterms (Rep a) a) => a -> [a]
subterms :: (Generic a, GSubterms (Rep a) a) => a -> [a]
recursivelyShrink :: (Generic a, RecursivelyShrink (Rep
a)) => a -> [a]
shrinkNothing :: a -> [a]
shrinkList :: (a -> [a]) -> [a] -> [[a]]
shrinkMap :: Arbitrary a => (a -> b) -> (b -> a) -> b ->
[b]
shrinkMapBy :: (a -> b) -> (b -> a) -> (a -> [a]) -> b
-> [b]
shrinkIntegral :: Integral a => a -> [a]
shrinkRealFrac :: RealFrac a => a -> [a]
shrinkBoundedEnum :: (Bounded a, Enum a, Eq a) => a ->
[a]
```

```
shrinkDecimal :: RealFrac a => a -> [a]


class Arbitrary1 (f :: Type -> Type) where
    liftArbitrary :: Gen a -> Gen (f a)
    liftShrink :: (a -> [a]) -> f a -> [f a]
arbitrary1 :: (Arbitrary1 f, Arbitrary a) => Gen (f a)
shrink1 :: (Arbitrary1 f, Arbitrary a) => f a -> [f a]

class Arbitrary2 (f :: Type -> Type -> Type) where
    liftArbitrary2 :: Gen a -> Gen b -> Gen (f a b)
    liftShrink2 :: (a -> [a]) -> (b -> [b]) -> f a b ->
[f a b]
arbitrary2 :: (Arbitrary2 f, Arbitrary a, Arbitrary b)
=> Gen (f a b)
shrink2 :: (Arbitrary2 f, Arbitrary a, Arbitrary b) => f
a b -> [f a b]



data Gen a
choose :: Random a => (a, a) -> Gen a
chooseInt :: (Int, Int) -> Gen Int
chooseInteger :: (Integer, Integer) -> Gen Integer
chooseWord64 :: (Word64, Word64) -> Gen Word64
chooseInt64 :: (Int64, Int64) -> Gen Int64
chooseUpTo :: Word64 -> Gen Word64
chooseBoundedIntegral :: (Bounded a, Integral a) => (a,
a) -> Gen a
chooseEnum :: Enum a => (a, a) -> Gen a
chooseAny :: Random a => Gen a
oneof :: HasCallStack => [Gen a] -> Gen a
frequency :: HasCallStack => [(Int, Gen a)] -> Gen a
elements :: HasCallStack => [a] -> Gen a
growingElements :: HasCallStack => [a] -> Gen a
sized :: (Int -> Gen a) -> Gen a
getSize :: Gen Int
resize :: HasCallStack => Int -> Gen a -> Gen a
scale :: (Int -> Int) -> Gen a -> Gen a
suchThat :: Gen a -> (a -> Bool) -> Gen a
suchThatMap :: Gen a -> (a -> Maybe b) -> Gen b
suchThatMaybe :: Gen a -> (a -> Bool) -> Gen (Maybe a)
applyArbitrary2 :: (Arbitrary a, Arbitrary b) => (a -> b
-> r) -> Gen r
applyArbitrary3 :: (Arbitrary a, Arbitrary b, Arbitrary
c) => (a -> b -> c -> r) -> Gen r
applyArbitrary4 :: (Arbitrary a, Arbitrary b, Arbitrary
c, Arbitrary d) => (a -> b -> c -> d -> r) -> Gen r
listOf :: Gen a -> Gen [a]
listOf1 :: Gen a -> Gen [a]
vectorOf :: Int -> Gen a -> Gen [a]
vector :: Arbitrary a => Int -> Gen [a]
infiniteListOf :: Gen a -> Gen [a]
infiniteList :: Arbitrary a => Gen [a]
shuffle :: [a] -> Gen [a]
```

```
sublistOf :: [a] -> Gen [a]
orderedList :: (Ord a, Arbitrary a) => Gen [a]
arbitrarySizedIntegral :: Integral a => Gen a
arbitrarySizedNatural :: Integral a => Gen a
arbitrarySizedFractional :: Fractional a => Gen a
arbitrarySizedBoundedIntegral :: (Bounded a, Integral a)
=> Gen a
arbitraryBoundedIntegral :: (Bounded a, Integral a) =>
Gen a
arbitraryBoundedRandom :: (Bounded a, Random a) => Gen a
arbitraryBoundedEnum :: (Bounded a, Enum a) => Gen a
arbitraryUnicodeChar :: Gen Char
arbitraryASCIIChar :: Gen Char
arbitraryPrintableChar :: Gen Char
generate :: Gen a -> IO a
sample :: Show a => Gen a -> IO ()
sample' :: Gen a -> IO [a]



data Fun a b = Fun (a :-> b, b, Shrunk) (a -> b)
applyFun :: Fun a b -> a -> b
applyFun2 :: Fun (a, b) c -> a -> b -> c
applyFun3 :: Fun (a, b, c) d -> a -> b -> c -> d
pattern Fn :: (a -> b) -> Fun a b
pattern Fn2 :: (a -> b -> c) -> Fun (a, b) c
pattern Fn3 :: (a -> b -> c -> d) -> Fun (a, b, c) d

class Function a where
    function :: (a -> b) -> a :-> b
functionMap :: Function b => (a -> b) -> (b -> a) -> (a
-> c) -> a :-> c
functionShow :: (Show a, Read a) => (a -> c) -> a :-> c
functionIntegral :: Integral a => (a -> b) -> a :-> b
functionRealFrac :: RealFrac a => (a -> b) -> a :-> b
functionBoundedEnum :: (Eq a, Bounded a, Enum a) => (a
-> b) -> a :-> b
functionVoid :: (forall b. void -> b) -> void :-> c



class CoArbitrary a where
    coarbitrary :: a -> Gen b -> Gen b

genericCoarbitrary :: (Generic a, GCoArbitrary (Rep a))
=> a -> Gen b -> Gen b
variant :: Integral n => n -> Gen a -> Gen a
coarbitraryIntegral :: Integral a => a -> Gen b -> Gen b
coarbitraryReal :: Real a => a -> Gen b -> Gen b
coarbitraryShow :: Show a => a -> Gen b -> Gen b
coarbitraryEnum :: Enum a => a -> Gen b -> Gen b
(><) :: (Gen a -> Gen a) -> (Gen a -> Gen a) -> Gen a ->
Gen a

newtype Blind a = Blind {
    getBlind :: a
```

```
}
newtype Fixed a = Fixed {
    getFixed :: a
}
newtype OrderedList a = Ordered {
    getOrdered :: [a]
}
newtype NonEmptyList a = NonEmpty {
    getNonEmpty :: [a]
}
data InfiniteList a = InfiniteList {
    getInfiniteList :: [a]
    infiniteListInternalData :: InfiniteListInternalData
a
}
newtype SortedList a = Sorted {
    getSorted :: [a]
}
newtype Positive a = Positive {
    getPositive :: a
}
newtype Negative a = Negative {
    getNegative :: a
}
newtype NonZero a = NonZero {
    getNonZero :: a
}
newtype NonNegative a = NonNegative {
    getNonNegative :: a
}
newtype NonPositive a = NonPositive {
    getNonPositive :: a
}
newtype Large a = Large {
    getLarge :: a
}
newtype Small a = Small {
    getSmall :: a
}
data Smart a = Smart Int a
newtype Shrink2 a = Shrink2 {
    getShrink2 :: a
}



data Shrinking s a = Shrinking s a
class ShrinkState s a where
    shrinkInit :: a -> s
    shrinkState :: a -> s -> [(a, s)]

newtype ASCIIString = ASCIIString {
    getASCIIString :: String
}
newtype UnicodeString = UnicodeString {
```

```
    getUnicodeString :: String

}
newtype PrintableString = PrintableString {
    getPrintableString :: String
}



data Property
class Testable prop where
    property :: prop -> Property
    propertyForAllShrinkShow :: Gen a -> (a -> [a]) ->
(a -> [String]) -> (a -> prop) -> Property

forAll :: (Show a, Testable prop) => Gen a -> (a ->
prop) -> Property
forAllShrink :: (Show a, Testable prop) => Gen a -> (a
-> [a]) -> (a -> prop) -> Property
forAllShow :: Testable prop => Gen a -> (a -> String) ->
(a -> prop) -> Property
forAllShrinkShow :: Testable prop => Gen a -> (a -> [a])
-> (a -> String) -> (a -> prop) -> Property
forAllBlind :: Testable prop => Gen a -> (a -> prop) ->
Property
forAllShrinkBlind :: Testable prop => Gen a -> (a ->
[a]) -> (a -> prop) -> Property
shrinking :: Testable prop => (a -> [a]) -> a -> (a ->
prop) -> Property
(==>) :: Testable prop => Bool -> prop -> Property

data Discard = Discard
discard :: a

(===) :: (Eq a, Show a) => a -> a -> Property
(=/=) :: (Eq a, Show a) => a -> a -> Property
total :: NFData a => a -> Property

ioProperty :: Testable prop => IO prop -> Property
idempotentIOProperty :: Testable prop => IO prop ->
Property

verbose :: Testable prop => prop -> Property
verboseShrinking :: Testable prop => prop -> Property
noShrinking :: Testable prop => prop -> Property
withNumTests :: Testable prop => Int -> prop -> Property
withMaxSuccess :: Testable prop => Int -> prop ->
Property
within :: Testable prop => Int -> prop -> Property
discardAfter :: Testable prop => Int -> prop -> Property
withDiscardRatio :: Testable prop => Int -> prop ->
Property
withMaxSize :: Testable prop => Int -> prop -> Property
withMaxShrinks :: Testable prop => Int -> prop ->
Property
```

```
once :: Testable prop => prop -> Property
again :: Testable prop => prop -> Property
mapSize :: Testable prop => (Int -> Int) -> prop ->
Property
(.&.) :: (Testable prop1, Testable prop2) => prop1 ->
prop2 -> Property
(.&&.) :: (Testable prop1, Testable prop2) => prop1 ->
prop2 -> Property
conjoin :: Testable prop => [prop] -> Property

data Every = Testable p => Every {
    getEvery :: p
}
(.||.) :: (Testable prop1, Testable prop2) => prop1 ->
prop2 -> Property
disjoin :: Testable prop => [prop] -> Property
data Some = Testable p => Some {
    getSome :: p
}


data Witness = (Typeable a, Show a) => Wit a
witness :: (Typeable a, Show a, Testable prop) => a ->
prop -> Property
coerceWitness :: Typeable a => Witness -> a
castWitness :: Typeable a => Witness -> Maybe a
counterexample :: Testable prop => String -> prop ->
Property
printTestCase :: Testable prop => String -> prop ->
Property


withProgress :: Testable prop => (TestProgress -> IO ())
-> prop -> Property
whenFail :: Testable prop => IO () -> prop -> Property
whenFail' :: Testable prop => IO () -> prop -> Property
expectFailure :: Testable prop => prop -> Property

label :: Testable prop => String -> prop -> Property
collect :: (Show a, Testable prop) => a -> prop ->
Property                       -- collect x = label (show
x)
classify :: Testable prop => Bool -> String -> prop ->
Property
tabulate :: Testable prop => String -> [String] -> prop
-> Property

cover :: Testable prop => Double -> Bool -> String ->
prop -> Property
coverTable :: Testable prop => String -> [(String,
Double)] -> prop -> Property
checkCoverage :: Testable prop => prop -> Property
checkCoverageWith :: Testable prop => Confidence -> prop
-> Property
```

```
data Confidence = Confidence {
    certainty :: Integer
    tolerance :: Double
}
stdConfidence :: Confidence
labelledExamples :: Testable prop => prop -> IO ()
labelledExamplesWith :: Testable prop => Args -> prop ->
IO ()
labelledExamplesWithResult :: Testable prop => Args ->
prop -> IO Result
labelledExamplesResult :: Testable prop => prop -> IO
Result
```

Test.QuickCheck.Monadic

```
newtype PropertyM (m :: Type -> Type) a = MkPropertyM {
    unPropertyM :: (a -> Gen (m Property)) -> Gen (m
Property)
}

run :: Monad m => m a -> PropertyM m a
assert :: forall (m :: Type -> Type). Monad m => Bool ->
PropertyM m ()
assertWith :: forall (m :: Type -> Type). Monad m =>
Bool -> String -> PropertyM m ()
pre :: forall (m :: Type -> Type). Monad m => Bool ->
PropertyM m ()
wp :: Monad m => m a -> (a -> PropertyM m b) ->
PropertyM m b
pick :: forall (m :: Type -> Type) a. (Monad m, Show a)
=> Gen a -> PropertyM m a
forAllM :: forall (m :: Type -> Type) a b. (Monad m,
Show a) => Gen a -> (a -> PropertyM m b) -> PropertyM m
b
monitor :: forall (m :: Type -> Type). Monad m =>
(Property -> Property) -> PropertyM m ()
stop :: forall prop (m :: Type -> Type) a. (Testable
prop, Monad m) => prop -> PropertyM m a

monadic :: (Testable a, Monad m) => (m Property ->
Property) -> PropertyM m a -> Property
monadic' :: (Testable a, Monad m) => PropertyM m a ->
Gen (m Property)
monadicIO :: Testable a => PropertyM IO a -> Property
monadicST :: Testable a => (forall s. PropertyM (ST s)
a) -> Property
runSTGen :: (forall s. Gen (ST s a)) -> Gen a

assertException :: Exception exc => (exc -> Bool) -> a
-> Property
assertExceptionIO :: Exception exc => (exc -> Bool) ->
IO a -> Property
assertDeepException :: (NFData a, Exception exc) => (exc
```

```
-> Bool) -> a -> Property
assertDeepExceptionIO :: (NFData a, Exception exc) =>
(exc -> Bool) -> IO a -> Property
```

Basic usage

```
classify (even x) "is even" $ ...
label ("even:" <> show (even x)) $ ...
collect x $ ...
tabulate "elements" (show <$> list) $ ...
```

```
-- e.g.
classify (even x) "even" $ ...
+++ OK, passed 100 tests; (53% even).


collect x $ ...
+++ OK, passed 100 tests;
56% 0                          -- value of x
17% 1
10% 2
 6% 3
 5% 4
 3% 5
```

***) usage: test an Arbitrary**
Use generate function

```
ghci> generate (arbitrary @Int)
26
ghci> generate (arbitrary @Int)
20
ghci> x <- generate (arbitrary @Int)
ghci> x
13
```

```
 ========================================================
                              ======
```

```
This
is
code
Monaco 14
```